

PÓS
UFG



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS



Desenvolvimento Web e Mobile - Full Stack

<https://pos.ufg.br/p/desenvolvimento-web-mobile-full-stack>

Programação Mobile com iOS - Prof. Silvano Malfatti

```
print("Sejam bem vindos!")
```

REFERÊNCIA:

<https://books.apple.com/br/book/the-swift-programming-language-swift-5-7/id881256329>

1 - Introdução à linguagem Swift

Swift é uma linguagem de programação desenvolvida pela **Apple**, anunciada em **2014** durante a WWDC (Worldwide Developers Conference). Foi criada como uma alternativa moderna ao **Objective-C**, com o objetivo de tornar o desenvolvimento para os ecossistemas **iOS**, **macOS**, **watchOS** e **tvOS** mais seguro, eficiente e fácil de aprender. Neste roteiro estudaremos os principais recursos do **Framework Foundation da Apple**.

1.1 Tipos de dados primitivos

A linguagem Swift possui características de desenvolvimento de código modernos como os conceitos de **Type Inference** e **Safety**. Tais conceitos permitem que o tipo de uma variável possa ser inferido sem a

necessidade de uma declaração explícita (embora esse modelo de declaração também seja suportado) e após a inferência de tipo mantém o tipo seguro, ou seja, não é possível alterar o tipo inferido pela variável em sua inicialização como demonstra o exemplo de código a seguir:

```
//Declaração implícita
var count = 0
let PI = 3.14
var name = "Swift"
var isOpen = true

//Declaração explícita
var version: Double = 1.3
var floor: Int = 6
let MAX: Int = 100
var country: String = "Brazil"
var isValid: Bool = false
var letter: Character = "X"
```

Código 1 - Tipos primitivos da linguagem Swift.



Em Swift até mesmo os tipos primitivos podem conter atributos e métodos.



1.2 Tipos de dados compostos (Tuplas, Arrays, Dicionários e Conjuntos)

- **Tuples:**

Uma tupla (Tuple) em Swift é uma estrutura de dados que pode armazenar múltiplos valores de diferentes tipos em um único conjunto. Diferente de arrays, que armazenam apenas valores do mesmo tipo, as tuplas permitem agrupar valores heterogêneos. Podem ser declaradas implicitamente ou explicitamente com rótulos ou sem. Exemplo:

```
//Tuplas
var car = ("Ferrari", 2015)
var explicitCar: (String, Int) = ("BMW", 2023)
var labelCar = (name: "BMW", year: 2023)
var explicitLabelCar: (name: String, year: Int) = (name: "BMW", year: 2023)
print("Carro: \(car.0) ano: \(car.1)")
print("Carro: \(labelCar.name) ano: \(labelCar.year)")
```

Código 2 - Tuplas

- **Arrays:**

Os arrays em Swift **não possuem tamanho fixo**, ou seja, podem crescer ou diminuir conforme necessário, desde que declarados como **var**.

```
//Arrays e Listas
var numeros: [Int] = [1, 2, 3]
numeros.append(4)
numeros.remove(at: 0)

let frutas: [String] = ["Maçã", "Banana", "Laranja"]
frutas.append("Uva") // Erro!
```

Código 3 - Arrays e listas dinâmicas.

- **Dicionários:**

Em Swift, um **dicionário** (**Dictionary<K, V>** ou simplesmente **[K: V]**), é uma coleção não ordenada de pares chave-valor, onde cada chave é única.

```
//Dicionarios
var dictCars: [String: Int] = ["BMW": 2023, "Audi": 2022]
var emptyDict: [String: String] = [:]
var anotherEmptyDict = [String: String]()
print(dictCars["BMW"] ?? 0)
dictCars["Mercedes"] = 2024
```

Código 4 - Dicionários.

- **Conjuntos:**

Em **Swift**, um **Set** (**Set<T>**) é uma coleção **não ordenada** de valores **únicos**. Ele é **mais eficiente** que um **Array** para **buscas**, pois utiliza uma estrutura baseada em hashing.

```

//Conjuntos
var setNumbers: Set = [1, 2, 3, 4, 5]
var emptySet = Set<String>()
let set1: Set = [1, 2, 3]
let set2: Set = [3, 4, 5]

let union = set1.union(set2)
print(union)

let intersection = set1.intersection(set2)
print(intersection)

let difference = set1.subtracting(set2)
print(difference)

struct Car: Hashable {
    let name: String
    let year: Int
}

let setA: Set<Car> = [
    Car(name: "BMW", year: 2020),
    Car(name: "Audi", year: 2021),
    Car(name: "Mercedes", year: 2022),
    Car(name: "Toyota", year: 2023),
    Car(name: "Honda", year: 2024)
]

let setB: Set<Car> = [
    Car(name: "Audi", year: 2021),
    Car(name: "Mercedes", year: 2022),
    Car(name: "Ford", year: 2023),
    Car(name: "Chevrolet", year: 2024),
    Car(name: "Tesla", year: 2025)
]

let intersec = setA.intersection(setB)
print("Interseção:", intersec)

```

Código 5 - Conjuntos com estruturas.

```

class ClassCar: Hashable {
    let name: String
    let year: Int

    //Construtor ou inicializador da classe
    init(name: String, year: Int) {
        self.name = name
        self.year = year
    }

    // Implementação de `Hashable` left hand side, right hand side
    static func == (lhs: ClassCar, rhs: ClassCar) -> Bool {
        return lhs.name == rhs.name && lhs.year == rhs.year
    }

    //Inout permite alterar a referencia de um objeto dentro da função
    func hash(into hasher: inout Hasher) {
        hasher.combine(name)
        hasher.combine(year)
    }
}

let car1 = Car(name: "BMW", year: 2020)
let car2 = Car(name: "Audi", year: 2021)
let car3 = Car(name: "Mercedes", year: 2022)
let car4 = Car(name: "Toyota", year: 2023)
let car5 = Car(name: "Honda", year: 2024)
let car6 = Car(name: "Audi", year: 2021)
let car7 = Car(name: "Mercedes", year: 2022)
let car8 = Car(name: "Ford", year: 2023)
let car9 = Car(name: "Chevrolet", year: 2024)
let car10 = Car(name: "Tesla", year: 2025)

let setCarA: Set<Car> = [car1, car2, car3, car4, car5]
let setCarB: Set<Car> = [car6, car7, car8, car9, car10]

let intersectCars = setCarA.intersection(setCarB)
print("Interseção:", intersectCars)

```

Código 6 - Conjuntos com classes.

1.3 Variáveis do tipo função:

Em **Swift**, uma **variável** pode **armazenar uma função**, permitindo que **funções sejam tratadas como valores**. Isso significa que podemos atribuir funções a variáveis, passá-las como argumentos e até mesmo retorná-las de outras funções.

```
//Variáveis do tipo função
func sum(val1: Int, val2: Int) -> Int {
    return val1 + val2
}

func dif(val1: Int, val2: Int) -> Int {
    return val1 - val2
}

func mul(val1: Int, val2: Int) -> Int {
    return val1 * val2
}

func div(val1: Int, val2: Int) -> Int {
    return val1 / val2
}

var operation: (Int, Int) -> Int = sum
operation = dif
operation = mul
operation = div

func execOperation(val1: Int, val2: Int, operation: (Int, Int)->Int) -> Int {
    return operation(val1, val2)
}

var result = execOperation(val1: 10 ,val2: 10, operation: sum)
print(result)

result = execOperation(val1: 10 ,val2: 10, operation: mul)
print(result)

result = execOperation(val1: 10 ,val2: 10, operation: dif)
print(result)

result = execOperation(val1: 10 , val2: 10, operation: div)
print(result)
```

Código 7 - variáveis do tipo função.

1.4 Variáveis opcionais

O conceito de Null Safety em Swift refere-se à maneira como a linguagem lida com valores opcionais (**Optionals**) para evitar erros comuns relacionados a null (**nil**), como *null pointer exceptions* em outras linguagens.

Em Swift, uma variável não pode ser **nil** a menos que seja explicitamente declarada como opcional (**Optional**). Isso força o programador a lidar com valores nulos de forma segura.

Em Swift, valores não opcionais (não Optional) não podem ser nil.

Exemplo:

```
//Variáveis opcionais
var text: String = "Swift"
//text = nil Erro!

var optional: String? = "Swift"
optional = nil

//Não irá quebrar a aplicação
print(optional?.uppercased())

//Realizando o Unwrapping com ??
print(optional ?? String())

//Realizando o Unwrapping com if let
if let secureText = optional {
    print(secureText)
}

printOptional(optionalText: optional)
func printOptional(optionalText: String?) {
    //Realizando o Unwrapping com guard let
    guard let secureText = optionalText else {
        return
    }
    print(secureText)
    print(optionalText!)
}
```

Código 8 - variáveis do tipo optional.

2 - Funções em Swift

Em Swift, as funções podem ser declaradas de diferentes maneiras, dependendo do contexto e da necessidade. Aqui estão os principais tipos de funções:

```
//Funções em Swift
func printSomething() {
    print("Hello World")
}
printSomething()

func showUserData(name: String, age: Int) {
    print("Me chamo \(name) e tenho \(age) anos.")
}
showUserData(name: "Swift", age: 11)

func pi() -> Double {
    return 3.1415
}
print(pi())

func classifyNumber(number: Int) -> (String, String) {
    let evenOrOdd = (number % 2 == 0) ? "Par" : "Impar"
    let positiveOrNegative = (number > 0) ?
    "Positivo" : (number < 0) ? "Negativo" : "Zero"
    return (evenOrOdd, positiveOrNegative)
}
print(classifyNumber(number: 9))
print(classifyNumber(number: -2))

func noLabelsFunc(_ name: String) {
    print(name)
}
noLabelsFunc("Swift")
```

Código 9 - Funções em Swift

```

func noLabelsFuncDefault(_ name: String = "Default") {
    print(name)
}
noLabelsFuncDefault("Swift")
noLabelsFuncDefault()

func funcDefaultWith(with name: String = "Default") {
    print(name)
}
funcDefaultWith(with: "Swift")
funcDefaultWith()

func varArgsFunc(varArgs: String...) {
    print(varArgs.sorted(by: { $0 < $1}))
}
varArgsFunc(varArgs: "Swift", "Java", "C#", "Python", "C++", "Dart", "Kotlin")

```

Código 10 - Funções em Swift

3 - Classes e Estruturas

Em Swift, uma **classe** (class) é um tipo de referência que define um modelo para a criação de objetos. Ela permite a organização de dados e funcionalidades em uma única estrutura reutilizável.

- Uma classe é um modelo para criar objetos que possuem propriedades e métodos.
- Ela suporta herança, permitindo que uma classe derive funcionalidades de outra.
- As classes são do tipo de referência, ou seja, quando um objeto de classe é atribuído a uma variável, a referência é compartilhada e não uma cópia é criada.

É importante salientar que por permitir o compartilhamento de referências, a liberação da memória para um objeto previamente alocado vai depender do número de referências válidas armazenadas pelo ARC.

Já uma struct (**estrutura**) é um tipo de dado que permite agrupar variáveis e comportamentos relacionados dentro de uma única unidade. Diferente de **class**, **struct** é um **tipo de valor** (value type), o que significa que cada instância criada é **copiada** ao invés de ser referenciada.

Característica	Struct (Estrutura)	Class (Classe)
Tipo de Referência	Tipo de valor (value type)	Tipo de referência (reference type)
Alocação na Memória	Stack (Pilha)	Heap (Heap)
Cópia de Objetos	Cópia independente (passagem por valor)	Referência compartilhada (passagem por referência)
Herança	❌ Não suporta	✅ Suporta
Deinitializers (deinit)	❌ Não possui	✅ Possui
Mutabilidade	let → Imutável, var → Mutável	Objetos de class podem ser mutáveis mesmo se a variável for let
Identidade do Objeto	Não possui identidade única	Possui identidade única
Thread Safety	Mais seguro para concorrência	Pode gerar problemas de concorrência devido à referência compartilhada

Tabela 1 - Diferenças entre classes e estruturas em Swift.

3.1 Classes

Uma classe em Swift pode conter um conjunto de propriedades e métodos utilizados respectivamente para armazenamento das informações do objeto e realização de ações sobre ele. Existem diferentes tipos de propriedades em uma classe:

- **Stored properties (propriedades armazenadas)**

São variáveis ou constantes que armazenam valores diretamente na instância da classe.

```
class Car {
    var brand: String = ""
    var year: Int = 0
}
```

Código 11 - Stored Properties em uma classe Swift.

- **Computed properties (propriedades computadas)**

Não armazenam um valor diretamente; em vez disso, fornecem um getter e um opcional setter para calcular dinamicamente um valor.

```
class Car {  
    //Stored properties  
    var brand: String = ""  
    var year: Int = 0  
  
    //Computed properties  
    var IPVA: Double = {  
        return year * 0.5  
    }  
}
```

Código 11 - Computed properties em Swift.

- **Property Observers**

Permitem monitorar mudanças em uma propriedade com `willSet` (antes da mudança) e `didSet` (após a mudança). Assim como as Stored Properties, as Observed Properties necessitam ser inicializadas.

```
class Car {  
    //Stored properties  
    var brand: String = ""  
    var year: Int = 0  
  
    //Computed properties  
    var IPVA: Double {  
        return 0.5 * Double(year)  
    }  
  
    //Static Properties  
    static var className: String = "Car"  
  
    //Observed properties  
    var color: String = "Black" {  
        willSet(newColor) {  
            print("A cor do carro será alterada para \(newColor)")  
        }  
        didSet {  
            print("A cor do carro foi alterada para \(oldValue) para \(color)")  
        }  
    }  
}
```

Código 11 - Property observes em Swift.

- **Inicializadores**

Servem para definir o estado inicial dos atributos de uma classe. Se uma classe não tem valores padrão para todas as propriedades, você precisa definir um `init` manualmente. Inicializadores podem ser do tipo designados ou convenientes. Inicializadores designados seguem a sintaxe padrão. Inicializadores convenientes deverão conter a palavra **convenience** para indicar que não inicializam todos os atributos em sua chamada e deverão chamar outros inicializadores para realizar esta tarefa.

```
//Initializers
convenience init() {
    self.init(brand: "", year: 0, color: "Black")
}

init (brand: String, year: Int,color: String) {
    self.brand = brand
    self.year = year
    self.color = color
}
```

Código 11 - inicializadores em Swift.

- **Deinicializadores**

Em Swift, o `deinit` (destrutor) é um método especial usado para executar ações quando uma instância de uma classe está sendo removida da memória. Ele é útil para liberar recursos, como conexões de banco de dados, observadores de notificações, arquivos abertos, etc. Cada classe contém no máximo um deinicializador.

```
deinit() {
    print("Este objeto foi desalocado")
}
```

Código 11 - Deinicializadores em Swift.

```
var fusca: Car? = Car(with: "Blue")
fusca = nil
```

Código 11: desalocando um objeto em Swift.

3.2 Herança e polimorfismo

```

class EletricCar: Car {
    var batteryCapacity: Int

    init (brand: String, year: Int, color: String, batteryCapacity: Int) {
        self.batteryCapacity = batteryCapacity
        super.init(brand: brand, year: year, color: color)
    }

    override
    //Computed properties
    var IPVA: Double {
        return 0
    }
}

class GasCar: Car {
    var gasTankCapacity: Int

    init (brand: String, year: Int, color: String, gasTankCapacity: Int) {
        self.gasTankCapacity = gasTankCapacity
        super.init(brand: brand, year: year, color: color)
    }
}

```

Código 11: herança e polimorfismo

3.3 Downcasting

O Downcasting é o processo de converter uma instância de uma superclasse para uma de sua subclasse. Isso acontece quando temos um objeto referenciado como um tipo mais genérico (por exemplo, `Car`), mas sabemos que ele é, na realidade, um tipo mais específico (por exemplo, `EletricCar`).

Em Swift, o Downcasting é feito usando os operadores `as?` e `as!`.

```

if (fusca is GasCar) {
    let gasCar = fusca as? GasCar
    print(gasCar?.gasTankCapacity ?? 0)
}

if (dolphin is EletricCar) {
    let eletricCar = dolphin as! EletricCar
    print(eletricCar.batteryCapacity)
}

```

Código 11: verificação de tipos e down casting.

4 - Retenção de memória em Ciclos

Em Swift, a retenção de memória em ciclos (retain cycles) ocorre quando dois ou mais objetos mantêm fortes referências um ao outro, impedindo que o ARC (Automatic Reference Counting) os desaloque da memória. Isso pode levar a vazamentos de memória porque os objetos nunca são liberados.

```
var fusca: Car? = GasCar(brand: "WF", year: 1970, color: "Black", gasTankCapacity: 30)
var dolphin: Car? = EletricCar(brand: "WF", year: 1970, color: "Black", batteryCapacity: 1000)

var proprietario1: Owner? = Owner(name: "Pedro", cpf: "345.868.943-43")
var proprietario2: Owner? = Owner(name: "Ana", cpf: "749.128.336-92")

fusca?.owner = proprietario1
dolphin?.owner = proprietario2

proprietario1?.car = fusca
proprietario2?.car = dolphin

fusca = nil
dolphin = nil

proprietario1 = nil
proprietario2 = nil
```

Código 12 - retenção de memória em ciclos.

Outra situação que devemos prestar atenção quanto aos ciclos de memória referem-se ao uso de **closures**. Os closures capturam fortemente as referências dos objetos, e se esse closure for armazenado em uma propriedade da mesma instância, ele pode criar um ciclo de retenção.

```

class Detran {
    var printCar: (()->Void)?
}

class ClosureCar {
    var name = "This is a car"

    init(detran: Detran?) {
        detran?.printCar = {
            print(self.name)
        }
    }

    deinit {
        print("ClosureCar desalocado")
    }
}

var detran:Detran? = Detran()
var closureCar:ClosureCar? = ClosureCar(detran: detran)
detran?.printCar?()
closureCar = nil

```

Código 12 - inicialização tardia de objetos.

5 - Protocolos

Um **protocolo** em Swift define um conjunto de **métodos, propriedades e requisitos** que uma classe, struct ou enum pode implementar. Ele funciona como um **contrato**, garantindo que qualquer tipo que adote o protocolo forneça as implementações exigidas.

- Define requisitos **sem implementar lógica**.
- Pode ser adotado por **classes, structs e enums**.
- Suporta **herança múltipla** (um tipo pode adotar vários protocolos).
- Permite **protocolos com implementações padrão** (via **extension**).

Possuem papel fundamental na programação de apps visto que muitos componentes visuais baseiam sua construção e comportamento no uso de protocolos. Podem ser utilizados também para garantir a ordem correta de passos para a construção de uma cena desenvolvida totalmente em código (ViewCodable).

```

protocol SceneProtocol {
    func addComponents()
    func setConstraints()
    func otherConfigs()
}

extension SceneProtocol {
    func createScene() {
        addComponents()
        setConstraints()
        otherConfigs()
    }
    func otherConfigs(){/*not implemented*/}
}

```

```

class SceneLogin: SceneProtocol {
    init() {
        createScene()
    }

    func addComponents() {
        print("Monta a hierarquia da cena")
    }

    func setConstraints() {
        print("Define o posicionamento dos componentes")
    }

    func otherConfigs() {
        print("Configurações adicionais")
    }
}

```

```

class SceneProfile {
    init() {
        createScene()
    }
}

extension SceneProfile: SceneProtocol {
    func addComponents() {
        print("Monta a hierarquia da cena")
    }

    func setConstraints() {
        print("Define o posicionamento dos componentes")
    }

    func otherConfigs() {
        print("Configurações adicionais")
    }
}

```

Código 13 - uso de protocolos para construção de cenas com ViewCodable.

6 - Delegates e DataSources em Swift

Os conceitos de **Delegate** e **DataSource** em Swift são padrões de design baseados no uso de **protocolos** para delegar responsabilidade a outras classes. Esses padrões são amplamente utilizados no UIKit.

6.1 Delegate e DataSource

```

class SceneMenu: ListDelegate, ListDataSource {
    func numItems() -> Int {
        return 5
    }

    func createItem(itemIndex: Int) {
        print("item \(itemIndex) criado")
    }

    func didSelectItem(itemIndex: Int) {
        print("item \(itemIndex) clicado")
    }
}

var list = ItenList()
var menu = SceneMenu()

list.dataSource = menu
list.delegate = menu
list.reloadList()
list.tapItem(index: 4)

```

Código 13 - Delegates e DataSources com protocolos.

7 - Enumerações

As enumerações (enums) em Swift são um tipo de dado poderoso e flexível que permite definir um conjunto de valores relacionados de forma organizada e segura. Diferente de muitas outras linguagens, em Swift, enums podem armazenar valores associados, suportar métodos, e até mesmo conformar-se a protocolos, tornando-os muito mais versáteis.

```
enum Resolution {  
    case ldpi  
    case mdpi  
    case hdpi  
    case xhdpi  
}  
  
var screenResolution: Resolution = Resolution.mdpi
```

Código 14 - Enumerações em Swift

8 - Generics

Generics em Swift permitem escrever código reutilizável e flexível, permitindo que funções, classes, estruturas e enumerações trabalhem com diferentes tipos de dados sem perder segurança e desempenho.

Generics são um dos recursos mais poderosos do Swift, permitindo que desenvolvedores escrevam código mais flexível, reutilizável e seguro. Eles são amplamente utilizados em estruturas nativas como [Array](#), [Dictionary](#) e [Set](#).

```
func swapGenericValues<T>(dataA: inout T, dataB: inout T) {  
    let copy = dataA  
    dataA = dataB  
    dataB = copy  
}  
  
var nameA = "Pedro"  
var nameB = "Maria"  
swapGenericValues(dataA: &nameA, dataB: &nameB)  
print(nameA)  
print(nameB)
```

Código 15 - Generics em Swift.

9 - Programação Concorrente

A concorrência permite que diferentes partes do código rodem simultaneamente, melhorando o desempenho de tarefas pesadas sem bloquear a interface do usuário. Em Swift, podemos implementar concorrência com:

1. GCD (Grand Central Dispatch)
2. Operation Queues
3. Swift Concurrency (async/await, Task, Actors)

9.1 GCD - iOS 4

O GCD é um framework da Apple que gerencia filas de tarefas para executar código de maneira assíncrona. O Grand Central Dispatch (GCD) gerencia a execução de tarefas assíncronas no iOS/macOS. O DispatchQueue é uma fila que gerencia quando e onde as tarefas serão executadas de forma serial ou concorrente. Pode ser utilizado de forma estática ou via instância. Outra característica do GCD é permitir que seja definida a prioridade de uma determinada tarefa assíncrona, o que vai determinar o tempo em que a tarefa deverá ser escalada para rodar na CPU. Processamentos paralelos podem ser executados na thread principal ou fora dela.

Existem dois tipos principais de filas (DispatchQueue):

Filas Globais:

- Fila Concorrente → Executa várias tarefas ao mesmo tempo, sem garantir ordem. Permitem definir a qualidade do serviço através do parâmetro **qos**.

◆ **Qualidade de Serviço (QoS – Quality of Service)**

QoS	Prioridade	Uso Ideal
<code>.userInteractive</code>	🚀 Muito Alta	Animações, gestos e UI imediata
<code>.userInitiated</code>	🔥 Alta	Processos rápidos iniciados pelo usuário
<code>.default</code>	⚖️ Média	Tarefas normais
<code>.utility</code>	🕒 Baixa	Downloads, cálculos demorados
<code>.background</code>	🚶 Muito Baixa	Sincronizações, backups

Tabela 2 - definindo prioridades com o GCD.

```
func doShomething(completion: @escaping ()->Void) {
    DispatchQueue.global().async {
        sleep(2)
        completion()
    }
}
print("inicio")
doShomething {
    print("1 - Dados carregados com sucesso!")
}
doShomething {
    print("2 - Dados carregados com sucesso!")
}
doShomething {
    print("3 - Dados carregados com sucesso!")
}
print("fim")
readLine()
```

Código 16 - concorrência com GCD.

A fila da Main Thread:

Usada para atualizar a interface do usuário (UI).



Atualizar a interface na Main Thread evita crashes e problemas de renderização.



```
func doShomethingMain(completion: @escaping ()->Void) {
    DispatchQueue.global(qos: .userInitiated).async {
        sleep(2)
        DispatchQueue.main.async {
            completion()
        }
    }
}
```

Código 16 - concorrência com GCD.

Filas Personalizadas:

- Fila Serial (attributes: .serial) → Além do comportamento assíncrono, Permitem executar uma tarefa por vez, em ordem **(default)**.

```
let serialQueue = DispatchQueue(label: "com.example.serialQueue", qos: .userInitiated)
//let serialQueue = DispatchQueue(label: "com.example.serialQueue", qos: .userInitiated, attributes: .concurrent)
func doShomethingSync(completion: @escaping ()->Void) {
    serialQueue.async {
        sleep(2)
        completion()
    }
}
print("inicio")
doShomethingSync {
    print("1 - Dados carregados com sucesso!")
}
doShomethingSync {
    print("2 - Dados carregados com sucesso!")
}
doShomethingSync {
    print("3 - Dados carregados com sucesso!")
}
print("fim")
readLine()
```

Código 16 - concorrência com GCD.

Executando múltiplas tarefas:

Usado quando precisamos executar várias tarefas concorrentes e esperar que todas terminem antes de continuar.

```

struct ResultData {
    var data1: String?
    var data2: String?
}

func doShomethingGroup(completion: @escaping (ResultData) -> Void) {
    let group = DispatchGroup()
    let queue = DispatchQueue.global(qos: .userInitiated)
    let syncQueue = DispatchQueue(label: "com.sync.queue") // Garantir acesso seguro a variáveis compartilhadas
    var result = ResultData()

    group.enter()
    queue.async {
        sleep(2)
        syncQueue.sync { result.data1 = "Dados da tarefa 1" }
        print("Tarefa 1 ok...")
        group.leave()
    }

    group.enter()
    queue.async {
        sleep(4)
        syncQueue.sync { result.data2 = "Dados da tarefa 2" }
        print("Tarefa 2 ok...")
        group.leave()
    }

    group.notify(queue: DispatchQueue.global()) {
        completion(result)
    }
}

print("Inicio")

doShomethingGroup { result in
    print("🚦 Resultados: \(result)")
}

```

Código 16 - concorrência com GCD.

9.2 Operation Queues - iOS 2

As Operation Queues são uma abstração sobre GCD, oferecendo mais controle sobre a execução de tarefas.

Vantagens sobre GCD

- Permitem dependências entre operações.
- Suportam cancelamento de tarefas.
- Melhor reutilização e organização do código.

Recursos	OperationQueue	DispatchQueue
Execução Assíncrona	✓ Sim	✓ Sim
Execução Serial/Concorrente	✓ Sim (configurável)	✓ Sim (configurável)
Dependências	✓ Sim (<code>addDependency()</code>)	✗ Não
Cancelamento	✓ Sim (<code>cancel()</code>)	✗ Não
Estado de Execução	✓ Sim (<code>isExecuting</code> , <code>isFinished</code>)	✗ Não
Controle de Prioridade	✓ Sim (<code>queuePriority</code> , <code>QoS</code>)	✓ Sim (<code>QoS</code>)

Tabela 3 - Diferenças entre OperationQueue e DispatchQueue.

Resumidamente podemos entender que se for preciso mais controle sobre a execução de tarefas, use OperationQueue. Porém, se o objetivo for apenas de execução assíncrona simples, DispatchQueue pode ser suficiente.

```
func requestServerBlock(completion: @escaping(() -> Void)) {
    let queue = OperationQueue()

    let operationBlock1 = BlockOperation {
        sleep(3)
        print("Bloco 1")
    }
    operationBlock1.qualityOfService = .userInitiated

    let operationBlock2 = BlockOperation {
        sleep(1)
        print("Bloco 2")
    }
    operationBlock1.qualityOfService = .background

    let operationBlock3 = BlockOperation {
        sleep(2)
        print("Bloco 3")
    }
    operationBlock3.qualityOfService = .utility

    let completionBlock = BlockOperation {
        DispatchQueue.main.async {
            completion()
        }
    }

    operationBlock1.addDependency(operationBlock3)
    operationBlock3.addDependency(operationBlock2)
    completionBlock.addDependency(operationBlock1)

    queue.addOperations([operationBlock1, operationBlock2, operationBlock3, completionBlock], waitUntilFinished: false)
}

requestServerBlock{
    print("Fim")
}

RunLoop.main.run()
```

Código 17 - concorrência com OperationQueue.

Implementação de uma classe para consumo de serviços Rest:

```
import Foundation

class ServiceManager {

    private static let session = URLSession.shared
    private static let operationQueue = OperationQueue()

    enum HTTPMethod: String {
        case get = "GET"
        case post = "POST"
        case put = "PUT"
        case delete = "DELETE"
        case patch = "PATCH"
    }

    // MARK: - Generic Request
    static func request<T: Decodable, U: Encodable>(
        urlString: String,
        method: HTTPMethod,
        payload: U? = nil,
        completion: @escaping (Result<T, Error>) -> Void
    ) {
        let operation = BlockOperation {
            guard let url = URL(string: urlString) else { return }

            var request = URLRequest(url: url)
            request.httpMethod = method.rawValue
            request.setValue("application/json", forHTTPHeaderField: "Content-Type")

            if let payload = payload {
                guard let payloadData = try? JSONEncoder().encode(payload) else { return }
                request.httpBody = payloadData
            }

            session.dataTask(with: request) { data, response, error in
                if let error = error {
                    completion(.failure(error))
                    return
                }
                guard let data = data else { return }

                do {
                    let decodedData = try JSONDecoder().decode(T.self, from: data)
```

```

        completion(.success(decodedData))
    } catch let decodingError {
        completion(.failure(decodingError))
    }
    }.resume()
}
operationQueue.addOperation(operation)
}

static func request<T: Decodable>(
    urlString: String,
    method: HTTPMethod,
    completion: @escaping (Result<T, Error>) -> Void
) {
    request(urlString: urlString, method: method, payload: Optional<String>.none, completion:
completion)
}
}

```

11 - UserDefaults

O `UserDefaults` é um recurso do iOS utilizado para **armazenar dados simples e persistentes** de forma local. Ele é amplamente usado para guardar **preferências do usuário**, **configurações do app**, **flags de estado** ou **informações leves** que precisam ser mantidas entre sessões.

Principais características do UserDefaults:

- **Persistência automática:** os dados são salvos automaticamente em disco.
- **Leitura e escrita rápidas.**
- Ideal para **valores pequenos** como `String`, `Int`, `Bool`, `Double`, `Array`, `Dictionary` e `Data`.
- A API é **sincronizada automaticamente**, mas também pode-se forçar com `synchronize()` (embora raramente seja necessário).

Implementação de uma classe para persistência de dados utilizando UserDefaults:

```
import Foundation

public final class StoreManager {

    // MARK: - Singleton Instance
    public static let shared = StoreManager()

    private var defaults: UserDefaults

    private init() {
        defaults = UserDefaults.standard
    } // Impede a criação de instâncias externas

    // MARK: - Métodos de Armazenamento

    /// Salva um valor (String) no UserDefaults para uma chave específica.
    func save(_ value: String, forKey key: String) {
        defaults.set(value, forKey: key)
    }

    /// Recupera um valor (String) do UserDefaults baseado na chave, retornando nil se não existir.
    func get(forKey key: String) -> String? {
        return defaults.string(forKey: key)
    }

    /// Verifica se um valor existe para a chave informada.
    func exists(forKey key: String) -> Bool {
        return defaults.object(forKey: key) != nil
    }

    /// Remove um valor do UserDefaults baseado na chave.
    func remove(forKey key: String) {
        defaults.removeObject(forKey: key)
    }
}
```

12 - Conclusões

O **Foundation** é um dos frameworks essenciais da Apple para o desenvolvimento de aplicações iOS, macOS, watchOS e tvOS. Ele faz parte da Foundation Framework, que fornece classes e estruturas fundamentais para

manipulação de dados, gerenciamento de tempo, tratamento de erros, manipulação de arquivos, redes, entre outras funcionalidades essenciais para qualquer aplicação.

Principais funcionalidades do Foundation:

- Manipulação de Strings e Coleções: Classes como String, Array, Set e Dictionary são amplamente utilizadas para manipulação de dados.
- Gerenciamento de Tempo: Date, DateFormatter e Calendar auxiliam no trabalho com datas e horários.
- Trabalho com Arquivos: FileManager permite acessar e modificar arquivos no sistema de arquivos do dispositivo.
- Redes e Comunicação: URLSession é usada para realizar requisições HTTP e interagir com APIs.
- Persistência de Dados: UserDefaults permite armazenar pequenas configurações de usuário de forma simples.
- Multithreading e Concorrência: OperationQueue e DispatchQueue são utilizadas para executar tarefas assíncronas e melhorar a performance.

O Foundation é essencial para o desenvolvimento de qualquer aplicação, pois fornece a base para lidar com dados e recursos do sistema.